

HONEYSHIELD

Real-Time Honeypot & Threat Intelligence Platform

- SSH + HTTP + FTP Honeypots
- MITRE ATT&CK Technique Mapping
- ML Anomaly Detection
- Real-Time WebSocket Dashboard
- IP Threat Reputation Scoring
- Coordinated Campaign Detection

Python 3.11 | Flask 3.0 | scikit-learn 1.4 | SQLite | Docker

v2.0 RELEASE

MIT License | Open Source

Cybersecurity Research Portfolio

Report generated March 28, 2026 at 18:59



CONFIDENTIAL - For Authorized Use Only

01 EXECUTIVE SUMMARY

HoneyShield is an open-source honeypot and threat intelligence platform designed to attract, capture, and analyse real-world cyberattacks. The system emulates three network services - SSH, HTTP (Apache/WordPress), and FTP - on publicly accessible ports, recording every interaction by automated scanners and malicious actors.

Unlike passive monitoring tools, a honeypot has no legitimate users, meaning every connection is inherently adversarial. This makes honeypots ideal for building ground-truth datasets, detecting emerging attack campaigns, and validating machine learning intrusion detection models.

6,214

Total Attacks

23

Unique IPs

3,414

SSH Attempts

2,400

HTTP Probes

400

FTP Attempts

02 SYSTEM ARCHITECTURE

HoneyShield follows a four-layer architecture designed for modularity, extensibility, and zero-dependency data persistence:

Layer	Components
Honeypot Layer	ssh_honeypot.py (paramiko) http_honeypot.py (stdlib) ftp_honeypot.py
Intelligence Layer	geoip/locator.py intelligence/mitre.py intelligence/reputation.py
ML Layer	ml/detector.py (scikit-learn IsolationForest + feature extraction)
Presentation Layer	Flask 3.0 Flask-SocketIO Leaflet.js Chart.js Tailwind CSS
Data Layer	SQLite WAL mode 3 tables 6 indexes parameterised queries
Alerting	Telegram Bot API SMTP (Gmail / custom relay)
Deployment	Docker + docker-compose environment-variable configuration

03 TECHNOLOGY STACK

Component	Technology	Justification
SSH emulation	paramiko 3.4	Full server-mode API, RSA key exchange
HTTP server	stdlib http.server	Zero dependencies, full response control
FTP server	raw sockets	Custom protocol handler, zero dependencies
Database	SQLite 3 + WAL	Zero-config, concurrent R/W, portable
GeoIP	ip-api.com	Free, no key, 45 req/min, cached 1h
Web framework	Flask 3.0	Lightweight, well-known, extensible

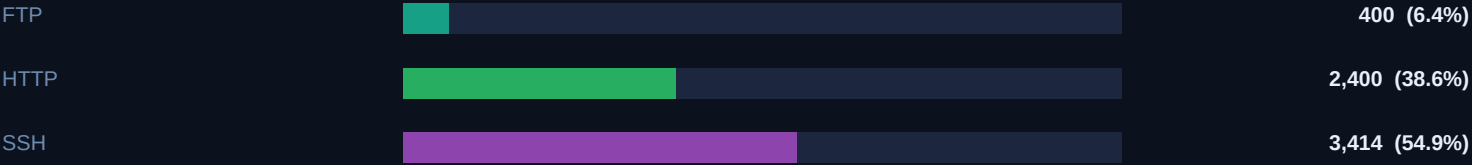
Real-time	Flask-SocketIO	WebSocket room broadcasting
Frontend	Tailwind + Chart.js + Leaflet	Modern, CDN, zero build step
ML detection	scikit-learn IsolationForest	Unsupervised, no labelled data needed
MITRE mapping	Custom rule engine	12 techniques, 6 tactics covered
Containerisation	Docker + Compose	Reproducible, port remapping

04

ATTACK ANALYSIS & RESULTS

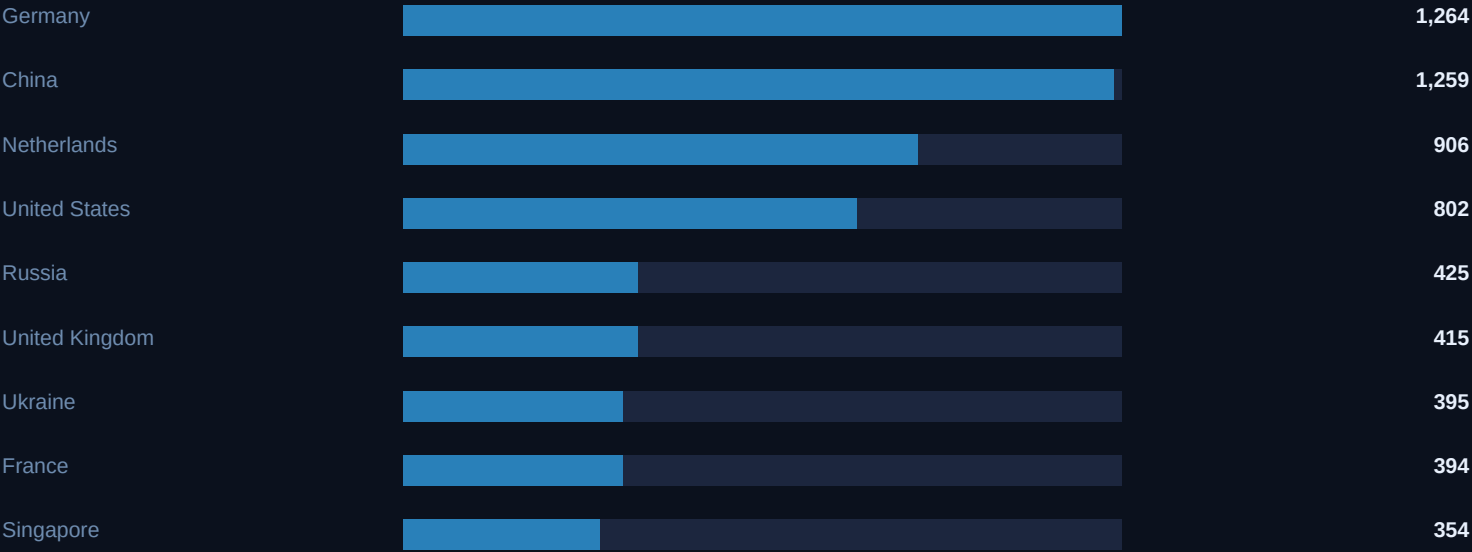
4.1

Attack Distribution by Service



4.2

Top Attacker Countries



4.3

Most-Tried SSH Credentials

Password	Attempt Count
raspberry	361 attempts
welcome	147 attempts
master	146 attempts
sunshine	144 attempts
shadow	144 attempts
toor	141 attempts
monkey	141 attempts
letmein	141 attempts

05

MITRE ATT&CK MAPPING

Each attack event is automatically tagged with one or more MITRE ATT&CK Enterprise technique IDs using payload classification rules. This enables correlation with known adversary TTPs and compatibility with SIEM/SOAR platforms.



T1110.001 Brute Force: Password Guessing



			2,214
T1133	External Remote Services		506
T1110.003	Brute Force: Password Spraying		464
T1190	Exploit Public-Facing Application		371
T1083	File and Directory Discovery		253
T1078	Valid Accounts		248
T1046	Network Service Discovery		222
T1595.001	Active Scanning: Scanning IP		222
T1059.004	Command and Scripting Interpr		135
T1212	Exploitation for Credential Access		125

06

ML ANOMALY DETECTION

HoneyShield uses scikit-learn's IsolationForest algorithm to detect statistically unusual attack patterns. The model trains on-demand on accumulated attack history - no labelled data required (unsupervised).

- Feature vector per attack:
- hour_of_day (sin/cos encoded) - off-hours attacks score higher
 - payload_length (normalised) - unusually long = exploit attempt
 - service_id (categorical) - cross-service anomalies detected
 - is_default_credential (binary) - root/admin/pi credential combos
 - path_depth (normalised) - deep URL paths = directory traversal
 - has_special_chars (binary) - SQLi/shell injection indicators

CRITICAL		260
HIGH		732
MEDIUM		354
LOW		2,068
INFO		2,800

07

IP THREAT REPUTATION SCORING

Each attacker IP receives a composite threat score (0-100) based on attack volume, multi-vector targeting, exploit indicators, off-hours activity, and persistence across days.

CRITICAL (>=90) | HIGH (>=75) | MEDIUM (>=50) | LOW (>=25) | INFO (<25)

Top Threat Actors

91.108.56.130		75/100	HIGH	304 atk
134.209.24.3		75/100	HIGH	281 atk
45.33.32.156		75/100	HIGH	307 atk
185.107.80.202		75/100	HIGH	329 atk
159.89.49.202		75/100	HIGH	291 atk
178.62.194.226		75/100	HIGH	330 atk
162.55.36.1		75/100	HIGH	327 atk
51.254.25.115		75/100	HIGH	310 atk
46.4.68.84		75/100	HIGH	309 atk
185.220.101.45		75/100	HIGH	343 atk

08

SECURITY DESIGN & ETHICS

The honeypot is engineered to be safe for the operator by design:

- Paramiko ALWAYS rejects authentication - no attacker gains shell access
- HTTP responses are static/hardcoded - no server-side code execution
- FTP always returns 530 Login incorrect - no filesystem access granted
- All DB inserts use parameterised queries - no SQL injection risk
- Dashboard protected by session-based authentication
- .env and data/ are in .gitignore - no credential leakage
- GeoIP queries use free-tier ip-api.com - minimal PII exposure

Ethical note: Deploy only on infrastructure you own and control. This system is a research tool for understanding attacker behaviour. All fake credentials in honeypot responses are obviously synthetic.



FUTURE WORK

Enhancement	Description
Distributed deployment	Multi-region network with Kafka + ClickHouse aggregation
Full shell emulation	Integrate Cowrie for complete SSH session recording
PCAP capture	Scapy/libpcap for packet-level recording and analysis
CVE correlation	Automatic CVE database lookup for detected payloads
Threat sharing	STIX/TAXII export for IOC sharing with threat intel communities
Active ML retraining	Periodic model retraining as new attack patterns emerge
PostgreSQL migration	Scalable DB backend for high-volume production deployments

10

REFERENCES

1. Spitzner, L. (2002). Honeypots: Tracking Hackers. Addison-Wesley Professional.

2. Provos, N. & Holz, T. (2007). Virtual Honeypots. Addison-Wesley.

3. Microsoft (2023). Microsoft Digital Defense Report 2023.

4. MITRE Corporation (2024). ATT&CK Framework v14. <https://attack.mitre.org>

5. OWASP Foundation (2021). OWASP Top Ten. <https://owasp.org>

6. Liu, F.T., Ting, K.M., & Zhou, Z.-H. (2008). Isolation Forest. IEEE ICDM.

7. Nawrocki, M. et al. (2016). A Survey on Honeypot Software. arXiv:1608.06249.

8. Vetterl, A. & Clayton, R. (2019). Honware: A Virtual Honeypot Framework. IEEE S&P.